

Corrective Retrieval-Augmented Generation (CRAG)

Design and Prototype Implementation Report

1. Introduction

Retrieval-Augmented Generation (RAG) systems depend on the quality of the information retrieved before an answer is generated. If the retrieved context is incomplete or irrelevant, the language model may produce an unsupported answer. Corrective Retrieval-Augmented Generation (CRAG) adds a verification stage that evaluates retrieved evidence before generation.

In the project architecture, CRAG is positioned after retrieval/reranking and before generation. Its role is to determine whether the retrieved context is sufficient and, when necessary, trigger another retrieval attempt.

2. Objective

- Evaluate whether retrieved chunks contain enough information to answer a query.
- Prevent insufficient retrieval results from proceeding directly to generation.
- Retry retrieval when the initial context is judged insufficient.
- Keep the CRAG component independent from the specific retrieval implementation.
- Provide a prototype that can later be connected to the project's real retriever.

3. CRAG Workflow



Sufficient: the validated context can continue to generation. **Insufficient:** the service performs another retrieval attempt and evaluates the new context.

4. Implementation

The CRAG component was implemented as a retriever-agnostic Python service. The main modules are:

File	Purpose
<code>crag/models.py</code>	Defines <code>Chunk</code> , <code>RetrievalDecision</code> , and <code>EvaluationResult</code> .
<code>crag/prompts.py</code>	Contains the instructions used by the LLM to judge retrieval sufficiency.
<code>crag/evaluator.py</code>	Sends the query and retrieved chunks to the LLM and returns a structured evaluation.
<code>crag/service.py</code>	Coordinates retrieval, evaluation, and retry attempts.
<code>fake_retriever/fake_retriever.py</code>	Temporarily simulates insufficient and sufficient retrieval results.
<code>tests/</code>	Contains evaluator tests, end-to-end tests, and a synthetic evaluation dataset.

5. Main Interfaces

The implementation uses simple interfaces so that the temporary retriever can later be replaced without changing the CRAG orchestration.

Retriever:

```
retrieve(query: str) → list[Chunk]
```

Evaluator:

```
evaluate(query: str, chunks: list[Chunk]) → EvaluationResult
```

The evaluation result contains a decision (sufficient or insufficient) and a short reason.

6. Prototype Behavior

Because the real vector database, embeddings, and production retriever are being developed separately, a FakeRetriever is currently used for development. It deliberately returns insufficient chunks on the first call and sufficient chunks on the next call.

Example: For the question “Where was the CEO of Company A working before?”, the first retrieval contains unrelated information about a product, an office, and revenue. The evaluator should return **insufficient**. The second simulated retrieval contains the CEO's previous employer, so the evaluator should return **sufficient**.

7. Testing

The evaluator test checks the LLM judgment independently. The end-to-end test verifies the complete retrieve → evaluate → retry flow. A small synthetic dataset is also used to compare expected and predicted sufficient/insufficient decisions.

End-to-end test:

```
python -m tests.test_crag
```

Dataset evaluation:

```
python -m tests.evaluate_dataset
```

8. Current Limitations and Future Integration

- The current retriever is simulated and will be replaced by the project's real retrieval implementation.
- Corrective retrieval is currently simulated rather than intelligently determined from the evaluation result.
- Answer generation is intentionally outside the current CRAG component.
- The current evaluator uses a binary sufficient/insufficient decision.
- Future integration can connect CRAG to the project's real retriever, reranker, query optimization, and generation stages.

9. Conclusion

The implemented CRAG prototype establishes a reusable retrieval-evaluation layer for the project's RAG pipeline. It evaluates retrieved evidence using an LLM, produces a structured sufficiency decision, and retries retrieval when the initial evidence is inadequate. The design keeps retrieval and generation separate, allowing the prototype to be integrated with the project's real components as they become available.